

Mocking with `unittest.mock`: Mock, MagicMock, and Patching

Week 5 Lecture — SE 413/513: Software Testing

Dr. Kamrul Hasan

Grand Valley State University

Fall 2026

Learning Objectives

By the end of today's class, you should be able to:

- Explain why real dependencies make unit tests slow, fragile, or hard to control, and how mocking addresses this.
- Create and configure **Mock** and **MagicMock** objects using `return_value` and `side_effect`.
- Verify how a dependency was used with call assertions and call-introspection attributes.
- Explain the difference between **Mock** and **MagicMock**, and recognize when magic-method support actually matters.
- Use `spec` and `autospec` to keep a mock honest about the interface it replaces.
- Use `patch()` to replace a dependency during a test, and explain why the *patch target* matters.
- Recognize common mocking mistakes, including over-mocking and interaction assertions that make tests fragile.

Part 1

Recap & Why Mocking Is Needed

~6 minutes

Recap: From Test Doubles to Mocking

Week 4 introduced five kinds of **test double** — dummy, stub, fake, spy, mock — using a running `order_service` with three collaborators: a **payment service**, a **database**, and an **email service**. It also introduced **state** vs. **behavior verification**, and the classical vs. mockist schools of TDD.

Key Idea

Today assumes that vocabulary. If dummy/stub/fake/spy/mock, or “state vs. behavior verification,” don’t sound familiar, revisit the Week 4 material first — we will not re-derive it here.

Today goes deeper into *mocking* specifically: the mechanics of Python’s `unittest.mock`, its two mock classes (`Mock` and `MagicMock`), and `patch()`.

Why Real Dependencies Get in the Way

A unit test should exercise a small piece of behavior in isolation. Real dependencies routinely make that difficult, because they can be:

- **slow** — a network round-trip or disk access on every test run
- **expensive** — a real payment charge, a paid API call
- **nondeterministic** — the current time, random values, race conditions
- **external** — outside your control, possibly unavailable in CI
- **side-effecting** — sending a real email, writing a real file

Mocking replaces a dependency like this with an object you fully control, while still exercising the real code that talks to it.

Part 2

Creating and Configuring Mocks

~14 minutes

Configuring return_value

Recall place_order() from Week 4:

```
def place_order(payment_service):  
    if payment_service.process_payment(100):  
        return "Order confirmed"  
    return "Payment failed"
```

Week 4 used a bare Mock() only to verify a call happened. A mock can also be *configured* to return whatever value the test needs:

```
payment_service = Mock()  
payment_service.process_payment.return_value = True  
  
result = place_order(payment_service)  
assert result == "Order confirmed"  
payment_service.process_payment.assert_called_once_with(100)
```

Three Steps, Every Time

create the mock → configure its behavior → pass it to the code under test

This pattern recurs throughout mocking, regardless of which mock class or configuration option is used.

Tip

`return_value` does *not* depend on the arguments a mock was called with — `process_payment(100)` and `process_payment(99999)` return exactly the same configured value. Need different results for different inputs? Use `side_effect` next.

Simulating Failures with `side_effect`

Real payment services fail sometimes, and error-handling code deserves tests too. `side_effect` makes a mock *raise* instead of return:

```
payment_service = Mock()
payment_service.process_payment.side_effect = ConnectionError(
    "payment gateway unreachable"
)

def test_place_order_handles_gateway_failure():
    try:
        place_order(payment_service)
        assert False, "expected a ConnectionError"
    except ConnectionError:
        pass
```

side_effect: Function or Sequence

As a **function**, called instead of returning a canned value — useful when the result depends on the input:

```
def fee_for(amount):  
    return amount * 0.03  
  
payment_service.calculate_fee.side_effect = fee_for  
assert payment_service.calculate_fee(100) == 3.0
```

As a **list**, consumed one value per call — handy for a flaky dependency that fails once, then succeeds:

```
payment_service.process_payment.side_effect = [  
    ConnectionError("timeout"),  
    True,  
]
```

Part 3

Verifying Interactions

~8 minutes

A Mock Remembers How It Was Used

Beyond `assert_called_once_with(...)` from Week 4, a few attributes inspect calls directly:

Expression	Purpose
<code>.assert_called()</code>	Called at least once
<code>.assert_called_once()</code>	Called exactly once (any args)
<code>.assert_called_once_with(...)</code>	Exactly one call, with these args
<code>.call_count</code>	How many times it was called
<code>.call_args</code>	Arguments of the <i>most recent</i> call
<code>.mock_calls</code>	The full sequence of calls made

Inspecting Calls Directly

```
payment_service = Mock()
payment_service.process_payment.return_value = True

place_order(payment_service)

assert payment_service.process_payment.call_count == 1
print(payment_service.process_payment.call_args)    # call(100)
```

Common Pitfall

Don't verify every interaction simply because a mock makes it possible. Asserting on every call couples a test to *how* code is implemented, not *what* it does — the same fragility Week 4 raised about the mockist school. Assert what matters; let the rest go unchecked.

Part 4

Mock vs. MagicMock: Why Two Classes?

~18 minutes

Not Every Operation Is an Ordinary Method Call

Everything so far has used `Mock`, and it's been enough — `process_payment(100)` is an **ordinary method call**.

But Python implements some operations through **magic methods** (“dunder” methods): `len(obj)`, `obj[0]`, `item in obj`, iteration, and `with` blocks are not ordinary method calls at all.

Operations and Their Magic Methods

Operation	Underlying magic method
<code>len(obj)</code>	<code>obj.__len__()</code>
<code>obj[key]</code>	<code>obj.__getitem__(key)</code>
<code>item in obj</code>	<code>obj.__contains__(item)</code>
<code>for x in obj</code>	<code>obj.__iter__()</code>
<code>with obj:</code>	<code>obj.__enter__()</code> and <code>obj.__exit__()</code>

A plain Mock does *not* implement these — calling `len()` on one raises `TypeError`. `MagicMock` is a Mock subclass that pre-configures all of them.

Key Idea

Mock handles ordinary method calls. **MagicMock** handles ordinary method calls *and* Python's magic-method protocols. Neither is “better” — it depends on the interface the code under test actually uses.

Example: A Richer Database Interface

Week 4's FakeDatabase had only ordinary methods — a plain Mock (or fake) fit fine. But suppose the database also acts like a collection of orders:

```
def summarize_orders(database):  
    if len(database) == 0:  
        return "No orders yet"  
    return f"{len(database)} orders, most recent: {database[0]}"
```

`len(database)` and `database[0]` are magic-method operations — a MagicMock models this directly:

```
database = MagicMock()  
database.__len__.return_value = 3  
database.__getitem__.return_value = "Order #1042"  
  
assert summarize_orders(database) == "3 orders, most recent: Order #1042"
```

Membership and Iteration Follow the Same Pattern

```
database.__contains__.return_value = True
assert "customer_42" in database

database.__iter__.return_value = iter(
    ["Order #1040", "Order #1041"]
)
assert list(database) == ["Order #1040", "Order #1041"]
```

A context-manager example (mocking a with `database.connection()` as `cursor: block`) works the same way, using `__enter__`/`__exit__` — see the reading and the backup slides for the full worked example.

A Decision Rule

Does the code only call **ordinary methods** on the dependency? → Use Mock.

Does the code use `len()`, indexing, iteration, membership (`in`), or a `with` block? → Use MagicMock.

Not sure yet, or the interface is mixed? → MagicMock is a safe default, but prefer Mock when sufficient — it says more clearly what the test actually needs.

Part 5

spec/autospec and patch()

~16 minutes

An Unconstrained Mock Hides Bugs

```
payment_service = Mock()  
payment_service.proccess_payment(100)    # typo -- but Mock allows it silently
```

If the real `PaymentService` has no `proccess_payment`, this test would pass while production would raise `AttributeError`. Passing `spec` constrains the mock to the real interface:

```
class PaymentService:  
    def process_payment(self, amount):  
        ...  
  
payment_service = Mock(spec=PaymentService)  
payment_service.process_payment(100)    # OK  
payment_service.proccess_payment(100)    # raises AttributeError
```

autospec: One Step Further

`autospec` (most often `autospec=True` with `patch()`) also checks *call signatures* — calling a one-argument method with two arguments fails the same way it would against the real class.

Tip

`spec/autospec` don't make a mock “more correct” about behavior — they only catch interface mismatches (wrong name, wrong argument count). Cheap insurance; a good default habit.

The Problem: A Module-Level Function

Constructor injection (Week 4) works when *you* control how a collaborator is supplied. It doesn't work for a module-level function called directly, like `requests.get`:

```
# shipping.py
import requests

def get_shipping_status(order_id):
    response = requests.get(f"https://ship.example.com/track/{order_id}")
    return response.json()["status"]
```

A unit test should not make a real HTTP request.

The Fix: patch()

`unittest.mock.patch()` temporarily replaces an object for the duration of a test:

```
from unittest.mock import patch

@patch("shipping.requests.get")
def test_get_shipping_status(mock_get):
    mock_get.return_value.json.return_value = {"status": "In Transit"}

    status = get_shipping_status(1042)

    assert status == "In Transit"
    mock_get.assert_called_once_with(
        "https://ship.example.com/track/1042"
    )
```

Key Idea: Patch *Where It Is Looked Up*

Key Idea

Patch the name where it is *looked up*, not where it was originally defined. `shipping.py` does `import requests` and calls `requests.get(...)`, so `shipping` is where that name is looked up at call time — hence `@patch("shipping.requests.get")`, *not* `@patch("requests.get")`.

This is one of the most common sources of confusion when first learning `patch()`.

What `patch()` Gives You by Default

By default, `patch()` replaces the target with a **MagicMock** — which is why chaining `mock_get.return_value.json.return_value` works with no extra setup: `mock_get`, its `return_value`, and every attribute accessed on it are all `MagicMock` objects unless configured otherwise.

Passing `autospec=True` to `patch()` additionally constrains the replacement to the real function's signature — combining patching with the interface safety from `spec/autospec`.

Part 6

When to Mock, Pitfalls & Wrap-Up

~9 minutes

When *Not* to Mock

Mocking is not automatically the right choice. Consider a small, deterministic, in-process helper:

```
def apply_discount(amount, rate):  
    return amount * (1 - rate)
```

Little to gain from mocking `apply_discount()` — it's fast, has no external effects, and a real call is simpler than any mock configuration.

General principle: mock dependencies that are slow, external, nondeterministic, or expensive; use the real object for everything else. Same judgment as classical vs. mockist TDD (Week 4) — mocking mechanics don't change it, they just give you the tools.

Common Mistakes

Common Pitfall

Patching the wrong location. Patch the name as looked up by the module under test.

Common Pitfall

MagicMock everywhere, out of habit. Use Mock when ordinary methods suffice — it's clearer.

Common Pitfall

Over-specifying interactions. Asserting every call makes tests brittle to harmless refactors.

Common Pitfall

Testing the mock, not the code. Pair interaction assertions with an assertion on an observable result.

Common Pitfall

Skipping spec/autospec. Hides typos and interface drift.

Key Takeaways

- ❶ **Mocking replaces slow, external, or nondeterministic dependencies** so tests run fast and deterministically.
- ❷ **return_value and side_effect configure behavior**; call assertions and call_args/mock_calls verify interactions.
- ❸ **Mock handles ordinary methods**; **MagicMock adds magic methods** — choose based on the interface, not by default.
- ❹ **spec/autospec catch interface mismatches** an unconstrained mock would silently hide.
- ❺ **patch() replaces a name where it is looked up**, not where it was defined.
- ❻ **Mocking mechanics do not replace judgment.** Whether to mock, and what to verify, are the same design questions from Week 4.

Discussion Question

Quick Check — Think, Then Discuss

A teammate says: “Since MagicMock can do everything Mock can do, we should just always use MagicMock.” Using at least one concept from today, argue for or against this position.

Think for a minute, then let's discuss as a class.

What We Didn't Cover Today

In the interest of realistic pacing, the following are in the reading but were **not** covered in lecture — you are responsible for them as reading:

- The context-manager cursor example in full (reading, Section 7.2; `__enter__`/`__exit__` and nested mock configuration) — summarized here, full example in backup slides.
- The complete “Practical Mental Model” quote list (reading, Section 12).
- Practice Exercises 1–5 and Discussion Questions beyond today’s in-class question (reading, Sections 14–15).
- The full References/Bibliography (reading, Section 17) — consult the reading directly for citations.

Questions?

Next reading: see Blackboard for the Week 6 assignment

Dr. Kamrul Hasan | Grand Valley State University

Backup Slides

Reference material only — not part of the 75-minute lecture

Backup: The Context-Manager Cursor Example (Full)

Many real database libraries expose a cursor as a context manager:

```
def get_customer_name(database, order_id):  
    with database.connection() as cursor:  
        cursor.execute("SELECT customer FROM orders WHERE id = ?", (order_id,))  
        row = cursor.fetchone()  
    return row[0]
```

The with statement relies on `__enter__()` and `__exit__()`. A MagicMock makes this straightforward:

```
database = MagicMock()  
cursor = database.connection.return_value.__enter__.return_value  
cursor.fetchone.return_value = ("Alice",)  
  
assert get_customer_name(database, 1042) == "Alice"  
cursor.execute.assert_called_once_with(  
    "SELECT customer FROM orders WHERE id = ?", (1042,) )
```

Backup: Reading Nested Mock Chains

Tip

Read `database.connection.return_value.__enter__.return_value` from left to right: “calling `database.connection()` returns *this*; entering *that* as a context manager returns *this*.” Nested mock configuration always mirrors the chain of calls the production code actually makes.

Backup: A Practical Mental Model (Full)

When you see `Mock()` or `MagicMock()`, think: *“A programmable stand-in whose behavior I control and whose interactions I can inspect afterward.”*

When you see `mock.method.return_value = ...`, think: *“When the code calls this, hand it this result.”*

When you see `mock.method.side_effect = ...`, think: *“Make this interaction fail, or compute its result, in a way I control.”*

When you see `mock.__len__`, `mock.__getitem__`, or similar, think: *“The code is using a Python protocol here — this needs `MagicMock`.”*

When you see `@patch("module.name")`, think: *“Replace this name, where this module looks it up, for the duration of the test.”*

Backup: Remaining Practice Exercises (Homework)

- **Exercise 1:** Write a test where `process_payment` raises `ConnectionError`; verify `place_order()` propagates it (or returns a failure message, if you add error handling).
- **Exercise 2:** For five scenarios (an `EmailService.send()`, a `catalog[sku]` lookup, iterating shipping events, a `logger.info()` call, a `with connection: block`), decide Mock or MagicMock and justify.
- **Exercise 3:** Build `Mock(spec=PaymentService)` and demonstrate both a successful call and an `AttributeError` on a misspelled method.
- **Exercise 4:** Patch `get_shipping_status()` to simulate a "Delivered" response; verify both the result and the requested URL.
- **Exercise 5:** Argue for or against "just always use MagicMock."

Backup: Remaining Discussion Questions

- 1 Why is `return_value` insensitive to call arguments, and when does that matter?
- 2 When would `side_effect` be preferable to `return_value`?
- 3 Why can excessive call-assertion checking make a test fragile, even when the production code is correct?
- 4 Why does a plain `Mock` raise `TypeError` on `len()`, while `MagicMock` does not?
- 5 What specific problem does `spec` solve that an unconstrained mock cannot?
- 6 Why is the *location* passed to `patch()` more important than where a dependency was defined?
- 7 How does the `Mock/MagicMock` distinction relate to state-vs-behavior verification from Week 4?
- 8 (*Covered in class*) Is “just always use `MagicMock`” a good idea?
- 9 In what situation would you prefer a real object over any kind of mock?

Backup: Full Summary Table (from the Reading)

Expression	Purpose
<code>Mock()</code>	A configurable test double for ordinary method calls
<code>MagicMock()</code>	A Mock that also supports Python magic methods
<code>return_value</code>	Configure what a call returns
<code>side_effect</code>	Raise an exception, call a function, or return values from a sequence
<code>assert_called_once_with(...)</code>	Verify one call, with these arguments
<code>call_count</code> / <code>call_args</code> / <code>mock_calls</code>	Inspect how a mock was used
<code>spec</code>	Constrain a mock to a real class's attributes
<code>autospec</code>	<code>spec</code> , plus call-signature checking
<code>patch()</code>	Temporarily replace a dependency for a test

Backup: Mock vs. MagicMock Capability Comparison

	Mock	MagicMock
Ordinary methods, <code>return_value</code> , <code>side_effect</code> , call assertions	Yes	Yes
<code>len(obj)</code> , <code>obj[key]</code> , <code>in</code> , iteration, <code>with</code>	No (raises <code>TypeError</code>)	Yes
Best when	Interface is plain method calls	Interface uses a Python protocol